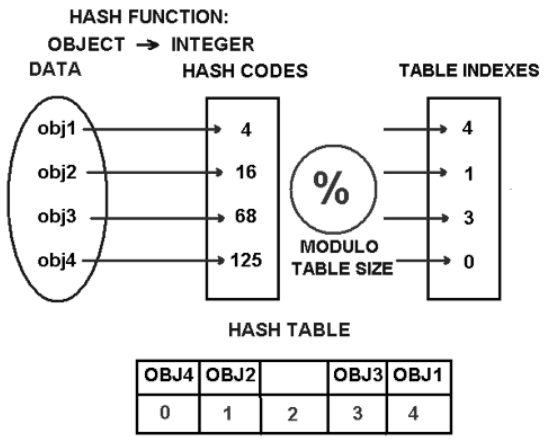
PKG 16: HASHING



`public int hashCode()`

Returns a hash code value for the object. This method is supported for the benefit of hash tables such as those provided by `HashMap`.

The general contract of `hashCode` is:

- Whenever it is invoked on the same object more than once during an execution of a Java application, the `hashCode` method must consistently return the same integer, provided no information used in `equals` comparisons on the object is modified. This integer need not remain consistent from one execution of an application to another execution of the same application.
- If two objects are equal according to the `equals(Object)` method, then calling the `hashCode` method on each of the two objects must produce the same integer result.
- It is *not* required that if two objects are unequal according to the `equals(java.lang.Object)` method, then calling the `hashCode` method on each of the two objects must produce distinct integer results. However, the programmer should be aware that producing distinct integer results for unequal objects may improve the performance of hash tables.

As much as is reasonably practical, the hashCode method defined by class `Object` does return distinct integers for distinct objects. (The hashCode may or may not be implemented as some function of an object's memory address at some point in time.)

Joshua Bloch's hashCode()

A short version

1. Create a `int result` and assign a **non-zero** value.
2. For *every field* `f` tested in the `equals()` method, calculate a hash code `c` by:
   - If the field f is a `boolean`: calculate `(f ? 0 : 1)`;
   - If the field f is a `byte`, `char`, `short` or `int`: calculate `(int)f`;
   - If the field f is a `long`: calculate `(int)(f ^ (f >>> 32))`;
   - If the field f is a `float`: calculate `Float.floatToIntBits(f)`;
   - If the field f is a `double`: calculate `Double.doubleToLongBits(f)` and handle the return value like every long value;
   - If the field f is an *object*: Use the result of the `hashCode()` method or 0 if `f == null`;
   - If the field f is an *array*: see every field as separate element and calculate the hash value in a *recursive fashion* and combine the values as described next.
3. Combine the hash value `c` with `result`:

   ```
   result = 37 * result + c
   ```

4. Return `result`

HASHING

```java
// A short implementation of hashCode() to demonstrate Joshua Bloch's approach
// @author Duc Ta PLUS

public class DisneyKeyChain {

    private static final int HASHCODE_NONZERO = 1;
    private static final int HASHCODE_PRIME = 37;

    private static int times_hashCodeMethodCalled = 0;
    private static int times_equalsMethodCalled = 0;

    private String name;
    private int price;

    public DisneyKeyChain(String name, int price) {
        this.name = name;
        this.price = price;
    }

    @Override
    public int hashCode() {
        DisneyKeyChain.alert_hashCodeMethodCalled();
        return hashCodeInt(this.price) + hashCodeString(this.name);
    }

    private static void alert_hashCodeMethodCalled() {
        DisneyKeyChain.times_hashCodeMethodCalled++;
        System.out.println("hashCode() called "
                + DisneyKeyChain.times_hashCodeMethodCalled + " time(s)");
    }

    private int hashCodeInt(int integer) {
        int hashCode = DisneyKeyChain.HASHCODE_NONZERO;
        int prime = DisneyKeyChain.HASHCODE_PRIME;
        hashCode = prime * hashCode + integer;
        return hashCode;
    }

    private int hashCodeString(String string) {
        return string.hashCode();
    }

    @Override
    public boolean equals(Object obj) {
        DisneyKeyChain.alert_equalsMethodCalled();
        if (obj instanceof DisneyKeyChain) {
            DisneyKeyChain keyChain = (DisneyKeyChain) obj;
            return (keyChain.name.equals(this.name)
                    && keyChain.price == this.price);
        } else {
            return false;
        }
    }

    private static void alert_equalsMethodCalled() {
        DisneyKeyChain.times_equalsMethodCalled++;
        System.out.println("equals() called "
                + DisneyKeyChain.times_equalsMethodCalled + " time(s)");
    }
}
```

HASHING

```
import java.util.HashMap;

public class DisneyKiosk {

    public static void main(String a[]) {

        HashMap<DisneyKeyChain, String> keyChainRack = new HashMap<>();
        DisneyKeyChain keyMickey = new DisneyKeyChain("Mickey", 10);
        DisneyKeyChain keyMinnie = new DisneyKeyChain("Minnie", 20);
        DisneyKeyChain keyGoofy = new DisneyKeyChain("Goofy", 30);

        System.out.println("\nAdding----------------------------------------");
        keyChainRack.put(keyMickey, "item#111");
        // hashCode() called 1 time(s)
        keyChainRack.put(keyMinnie, "item#222");
        // hashCode() called 2 time(s)
        keyChainRack.put(keyGoofy, "item#333");
        // hashCode() called 3 time(s)

        System.out.println("\nTest 1----------------------------------------");
        System.out.println("Hash code of the key:    " + keyMickey.hashCode());
        System.out.println("System identityHashCode: " + System.identityHashCode(keyMickey));
        // hashCode() called 4 time(s)
        // Hash code of the key:    -1990497529
        // System identityHashCode:  664223387

        System.out.println("Associated value of the key: " + keyChainRack.get(keyMickey));
        // hashCode() called 5 time(s)
        // Associated value of the key: item#111

        System.out.println("\nTest 2----------------------------------------");
        DisneyKeyChain keyMickey_2 = new DisneyKeyChain("Mickey", 10);
        System.out.println("Hashcode of the key:     " + keyMickey_2.hashCode());
        System.out.println("System identityHashCode: " + System.identityHashCode(keyMickey_2));
        // hashCode() called 6 time(s)
        // Hashcode of the key:     -1990497529
        // System identityHashCode: 122883338

        System.out.println("Associated value of the key: " + keyChainRack.get(keyMickey_2));
        // hashCode() called 7 time(s)
        // equals() called 1 time(s)
        // Associated value of the key: item#111

        System.out.println("\nTest 3----------------------------------------");
        DisneyKeyChain keyMickey_3 = new DisneyKeyChain("Mickey", 10);
        System.out.println("Hashcode of the key:     " + keyMickey_3.hashCode());
        System.out.println("System identityHashCode: " + System.identityHashCode(keyMickey_3));
        // hashCode() called 8 time(s)
        // Hashcode of the key:     -1990497529
        // System identityHashCode: 666641942

        keyChainRack.put(keyMickey_3, "item#444");
        // hashCode() called 9 time(s)
        // equals() called 2 time(s)

        System.out.println("Associated value of the key: " + keyChainRack.get(keyMickey_3));
        // hashCode() called 10 time(s)
        // equals() called 3 time(s)
        // Associated value of the key: item#444
    }
}
```

HASHING

```
Other implementations of HashCode()

// Simple Implementation
@Override
public int hashCode() {
    return (int) id * fName.hashCode() * lName.hashCode();
}


// Standard Implementation
@Override
public int hashCode() {
    int hash = 1;
    hash = 37 * hash + (int) id;
    hash = 37 * hash + (fName == null ? 0 : fName.hashCode());
    hash = 37 * hash + (lName == null ? 0 : lName.hashCode());
    return hash;
}


// IntelliJ IDEA
@Override
public int hashCode() {
    int result = (int) (id ^ (id >>> 32));
    result = 37 * result + fName.hashCode();
    result = 37 * result + lName.hashCode();
    return result;
}


// Eclipse
@Override
public int hashCode() {
    final int prime = 37;
    int result = 1;
    result = prime * result + ((fName == null) ? 0 : lName.hashCode());
    result = prime * result + (int) (id ^ (id >>> 32));
    result = prime * result + ((fName == null) ? 0 : lName.hashCode());
    return result;
}
```